



Phase 1 of 6

✓ Step 1.1 – Create your GitHub repo `user-service`

1. Go to github.com
2. Click on **New repository**
3. Name it: `user-service`
4. Keep it **Public or Private** (your choice)
5. Click **Create repository**
6. Copy the Git URL shown

Just know n keep

REST APIs generate doors or gateways to clients to access the different parts of our application like login, homepage, order page, cart page, service page, wish list page, about us page, etc..

REST APIs will be generated like `/users` , `/login` , etc

`nip.io` is a free wildcard DNS service that maps IP addresses to domain names automatically. It lets you access services using URLs like `app.<your-ip>.nip.io` without setting up a custom DNS.

Now on your system:

```
git clone https://github.com/YOUR_USERNAME/user-service.git
cd user-service
```

Folder Structure

Important

```
user-service/
├─ src/
│   └─ main/
│       └─ resources/
│           └─ templates/
│               └─ java/
│                   └─ com/example/users/
```

```
|
|
|
|
|
├── controller/
├── model/
├── service/
└── UserServiceApplication.java
├── Dockerfile
├── pom.xml
└── README.md
```

Part	Meaning
src	Source code folder of your project
main	Main application code (vs test code, which goes to <code>src/test</code>)
java	Indicates this is Java code
com	Part of a reverse domain name (e.g., <code>com.google</code> , <code>org.apache</code>)
example	Your organization or company name (e.g., <code>example.com</code>)
users	Your actual project or module name (like "users" service)

✅ Step 1.2 – Add Spring Boot Code I Gave

Copy these files/folders inside the `user-service` folder:

1. Create folder structure:

```
mkdir -p src/main/java/com/example/users/controller
mkdir -p src/main/java/com/example/users/model
mkdir -p src/main/java/com/example/users/service
mkdir -p src/main/resources
touch application.properties
```

2. Add files I gave you:

- `UserServiceApplication.java` → `src/main/java/com/example/users/`
- `User.java` → `model/`
- `UserService.java` → `service/`

- UserController.java → controller/
- pom.xml → root
- Dockerfile → root
- README.md → root

✅ How to Change Spring Boot Port to 8081

Since **Jenkins uses port 8080** by default, your Spring Boot app (user-service) should use a **different port**, like **8081**, to avoid conflict — especially when testing both locally.

1. create a file named `application.properties` here:

```
src/main/resources/application.properties
```

2. add this line:

```
server.port=8081
```

- ``/src/main/java/`` → for Java code
- ``/src/main/resources/`` → for config files (like ``application.properties``)
- They are separate but both needed in a Spring Boot app
- So create: ``src/main/resources/application.properties`` and add ``server.port=8081``

JAVA Springboot App's Coding Section - ✅ Step 1.2

For the codes of below java files refer the codes pdf

 UserServiceApplication.java

 User.java (Model)

 UserService.java

 UserController.java

 home.html

 pom.xml (Basic)

 Dockerfile

Then

Create a DockerHub Repository

1. Go to <https://hub.docker.com>
2. Create a new **Public** or **Private** repository named:

```
user-service
```

3. Example image name:

```
docker.io/kapilanramesh/user-service
```

| Replace `kapilanramesh` with your DockerHub username.

Then

From inside **user-service** folder,

Run :

```
docker login
docker tag user-service docker.io/kapilanramesh/user-service
docker push docker.io/kapilanramesh/user-service
```

✅ Step 1.3 – Build and Run the Service Locally

1. You must install **Maven** and **Java 17**. Run:

```
sudo apt update
sudo apt install maven openjdk-17-jdk -y
```

2. Then build the app:

```
mvn clean package
```

If successful, you'll see:

```
BUILD SUCCESS
```

3. Run it:

```
java -jar target/user-service-0.0.1-SNAPSHOT.jar
```

Now open your browser and check:

```
http://localhost:8081/users
```

✅ Step 1.4 – Commit & Push to GitHub

```
git add .  
git commit -m "Initial commit with Spring Boot user service"  
git push origin main
```

🔥 **Phase 2 of 6:

Docker + Jenkins CI Pipeline for your `user-service`. (It Involves CI Only)

You'll set up a Jenkins pipeline that:

1. Pulls your Spring Boot code from GitHub
 2. Builds it using Maven
 3. Builds a Docker image
 4. Pushes it to DockerHub
 5. Deploys later to Kubernetes (we'll do this in Phase 3+)
-

✅ Step 2.1 – Install Docker on Ubuntu (if not installed)

✅ Step 2.2 – Add Jenkins User to Docker Group (if needed)

✅ Step 2.3 – Jenkins Pipeline Script

Create a **new Pipeline job** in Jenkins :

- Name: `user-service-pipeline`
- Type: **Pipeline**
- Scroll to **Pipeline Script** section and paste:

```
pipeline {
    agent any

    environment {
        DOCKER_IMAGE = "kapilarnamesh/user-service"
    }

    stages {
        stage('Clone Code') {
            steps {
                git 'https://github.com/kapilarnamesh/user-service.git'
            }
        }

        stage('Build with Maven') {
            steps {
                sh 'mvn clean package'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t $DOCKER_IMAGE:latest .'
            }
        }

        stage('Push to DockerHub') {
            steps {
                withCredentials([usernamePassword(credentialsId: 'dockerhub-creds', usernameVariable: 'DOCKER_USER', passwordVariable: 'DOCKER_PASS')]) {
                    sh ""
```

```
-password-stdin      echo "$DOCKER_PASS" | docker login -u "$DOCKER_USER" -  
                      docker push $DOCKER_IMAGE:latest  
                      ""  
                    }  
                  }  
                }  
              }  
            }
```

✔ Step 2.4 – Create DockerHub Credentials in Jenkins

1. Go to Jenkins Dashboard → **Manage Jenkins** → **Credentials**
2. Under (global) → Add **Username & Password**
3. ID: `dockerhub-creds`
(same as in pipeline)
4. Username: your DockerHub username
5. Password: your DockerHub password

Once you run the Jenkins job:

- ✓ It will pull your GitHub repo
- ✓ Build the `.jar`
- ✓ Build the Docker image
- ✓ Push it to DockerHub

 To Test:

```
docker pull kapilanameash/user-service:latest
docker run -p 8081:8081 kapilanameash/user-service
```

📌 **Command for docker tag and push to dockerhub (know n keep)**

```
docker tag my-custom-app docker.io/kapilanramesh/user-service:latest
docker push docker.io/kapilanramesh/user-service:latest
```

🌟 Phase 3 of 6 :

EKS Cluster Setup + Helm + Kubernetes Deploy

✅ Phase 3: Kubernetes Deployment via Helm (Step-by-Step)

✅ Step 3.1: Install Helm (if not already installed)

Run this on your Ubuntu EC2 system:

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 |  
bash  
helm version
```

✅ Step 3.2: Create a Helm Chart for `user-service`

In your **Ubuntu** workspace:

```
mkdir -p user-service-helm  
cd user-service-helm  
helm create user-service
```

"**helm create user-service**" creates a basic Helm chart at: `user-service/`

📁 Folder Structure

```
user-service/  
├── Chart.yaml           # Info about the chart  
├── values.yaml          # Your config (image, port, etc.)  
└── templates/          # All k8s templates:  
    ├── deployment.yaml  # Deployment (pods, containers)  
    ├── service.yaml     # Service (NodePort, LoadBalancer)  
    └── _helpers.tpl     # Used for template variables
```

✅ Step 3.3: Edit the Templates and Values YAMLs

For the YAML scripts of below refer the codes pdf

✅ values.yaml

✅ deployment.yaml

✅ service.yaml

To Edit the YAML files (be inside "user-service-helm" Directory)

```
nano user-service/values.yaml
nano user-service/deployment.yaml
nano user-service/service.yaml
```

✅ Step 3.4: Deploy the App to EKS

Run this in Ubuntu workspace:

```
helm install user-service ./user-service
```

Wait for the deployment to finish:

```
kubectl get all
```

✅ Step 3.5: Access the App via LoadBalancer

Run:

```
kubectl get svc
```

Find the EXTERNAL-IP of the user-service service.

Visit in browser:

```
http://<EXTERNAL-IP>:8081/users
```

Once this works, you've completed:

✅ You should see your app's user list (empty by default).

🚀 CI → DockerHub → Kubernetes Deployment with Helm ✅

🔥 Phase 4 of 6 :

Kubernetes Monitoring with Prometheus + Grafana – a must-have for advanced DevOps projects.

#✅ Phase 4 – Monitoring Setup (Prometheus + Grafana) in EKS via Helm

This phase will:

- Monitor your **EKS cluster**, app pods, CPU/memory usage, and container metrics
- Setup dashboards using **Grafana**
- Make your DevOps project look super pro 🙌

🔧 Step-by-step Guide

◆ Step 1: Add the kube-prometheus Helm repo

```
helm repo add prometheus-community https://prometheus-  
community.github.io/helm-charts
```

```
helm repo update
```

Helm just **registers the chart repository** and **downloads its index file** (metadata, not the actual app chart). The **actual Prometheus/Grafana chart** gets downloaded **only when you install it**

◆ Step 2: Create a namespace for monitoring

```
kubectl create namespace monitoring
```

It keeps the cluster organized and avoids naming conflicts. It isolates monitoring components (like Prometheus, Grafana) from other app workloads.

◆ Step 3: Install Prometheus + Grafana using Helm

```
helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack \
  --namespace monitoring
```

⌚ Wait 1–2 mins. This will install:

- Prometheus server
- Grafana
- Node Exporter
- Kube State Metrics

◆ Step 4: Get Grafana admin password

```
kubectl get secret --namespace monitoring kube-prometheus-stack-grafana \
  -o jsonpath="{.data.admin-password}" | base64 --decode ; echo
```

You'll get a password like: eyJrIjoiZXhhb...

◆ Step 5: Access Grafana in browser (via port-forwarding)

```
kubectl port-forward --namespace monitoring svc/kube-prometheus-stack-grafana
3000:80
```

Or,

If you want to access Grafana via public IP without port forwarding, you must expose Grafana with `LoadBalancer`

```
kubectl edit svc kube-prometheus-stack-grafana -n monitoring
```

→ Change `type: ClusterIP` to `type: LoadBalancer`.

But this is **not recommended for security** unless you configure auth and firewall rules.

- Save and exit.
- Wait for 1–2 minutes, then run:

```
kubectl get svc -n monitoring
```

You'll see an **EXTERNAL-IP** — open `http://<EXTERNAL-IP>` in browser.

Login with:

- **Username:** `admin`
- **Password:** (the one from Step 4)

◆ Step 6: Import dashboards

Inside Grafana:

- Go to **Dashboards** → **Import**
- Use these IDs to import:
 - Kubernetes Cluster Monitoring: `315`
 - Node Exporter Full: `1860`
 - Spring Boot App Metrics: `6756` (if micrometer is used)

✅ **Phase 5 of 6:

Kubernetes Service Exposure (Web Access from Browser)

Let's expose your Spring Boot app running inside the EKS cluster to the internet.

🔥 Goal:

| Make your deployed web app accessible at `http://<EXTERNAL-IP>` in your browser.

✅ Step 1: Modify Helm Chart's `service.yaml`

Run this in your EC2 terminal (connected to EKS):

```
nano user-service/templates/service.yaml
```

Replace entire content with this:

```
apiVersion: v1
kind: Service
metadata:
  name: {{ include "user-service.fullname" . }}
  labels:
    {{- include "user-service.labels" . | nindent 4 }}
spec:
  type: LoadBalancer
  ports:
    - port: {{ .Values.service.port }}
      targetPort: {{ .Values.service.targetPort }}
      protocol: TCP
      name: http
  selector:
    {{- include "user-service.selectorLabels" . | nindent 4 }}
```

✅ Step 2: Update `values.yaml`

Open:

```
nano user-service/values.yaml
```

Make sure this section looks like this:

```
service:
  type: LoadBalancer
  port: 80
  targetPort: 8080
```

Save and exit.

✅ Step 3: Upgrade your Helm release

Run this to apply the changes:

```
helm upgrade user-service ./user-service
```

✅ Step 4: Get the External IP

Run:

```
kubectl get svc
```

Wait until `EXTERNAL-IP` appears (may take 1–2 minutes).

Then open in browser:

```
http://<EXTERNAL-IP>
```

🎉 Done!

Let me know once you're able to access the webpage. We'll then continue to **Phase 6: Ingress + Domain + SSL (Advanced)**.

🔥 **Phase 6 of 6:

Ingress Controller + Custom Domain Setup (Optional, but Powerful)

This step helps you expose your Spring Boot app with a clean **domain name** (like `user.kapilan.in`), instead of using long, random LoadBalancer URLs.

✅ Step-by-Step Plan:

✂ 1. Install NGINX Ingress Controller on EKS

We'll install it via Helm:

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx  
helm repo update
```

```
helm install ingress-nginx ingress-nginx/ingress-nginx \\  
  --namespace ingress-nginx --create-namespace
```

2. Get the Ingress Controller's External IP

```
kubectl get svc -n ingress-nginx
```

Output : "a9619eb49e34..."

Since nip.io works only with raw IP addresses, not DNS, we must convert this LoadBalancer DNS to its IP.

So use nslookup , (Get the public IP of your LoadBalancer DNS)

```
nslookup a9619d9eb49e3428b8b63b4aa5cf083a-742086938.us-east-1.elb.amazonaws.com
```

Output:

```
Name:      a9619d9eb49e3428b8b63b4aa5cf083a-742086938.us-east-1.elb.amazonaws.com
Address: 3.85.210.102
```

Copy that Address IP — let's say it's 3.85.210.102 . (Replace this IP in the place of EXTERNAL IP in values.yaml)

3. Update Your Domain DNS (if you have one)

If you own a domain (e.g., from GoDaddy, Namecheap), go to DNS settings and **add an A record**:

- **Host:** user (or @ if root domain)
- **Points to:** <EXTERNAL-IP from above>

🚫 If you don't have a domain, skip this step and use nip.io instead:

```
user-service.<EXTERNAL-IP>.nip.io
```

(Free wildcard DNS that maps to any IP)

4. Create Helm Ingress Configuration in values.yaml

Modify your `values.yaml` like this:

```
ingress:
  enabled: true
  className: "nginx"
  annotations:
    nginx.ingress.kubernetes.io/rewrite-target: /
  hosts:
    - host: user-service.<EXTERNAL-IP>.nip.io
      paths:
        - path: /
          pathType: Prefix
  tls: []
```

✅ Or replace `<EXTERNAL-IP>` with your actual IP/domain.

5. Redeploy Helm with Updated Ingress

```
helm upgrade --install user-service ./user-service
```

✅ 6. Test Your App

Now, open:

```
http://user-service.<EXTERNAL-IP>.nip.io
```

Your Spring Boot app should be accessible via this clean domain 🎉
